# Atelier Créactifs: Raspberry Pi

## Séance 3: Python et l'interface GPIO

François Marelli
UMONS, ISIA Lab

2 mars 2026

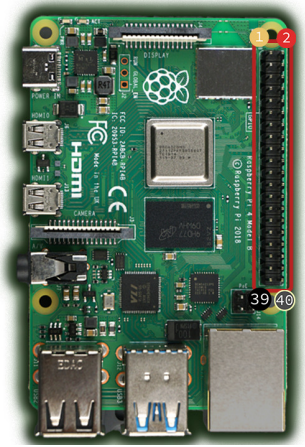
ATELIERS créactifs!

ISIA Lab
INFORMATION, SIGNAL, ARTIFICIAL INTELLIGENCE

UMONS
Université de Mons

# L’interface GPIO

*General Purpose Input/Output*

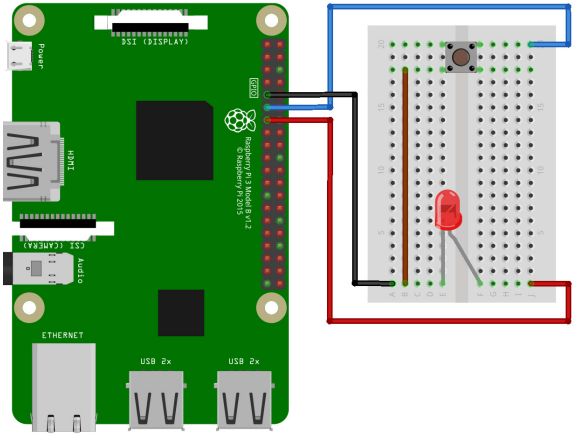



| | | | | | | |
|---|---|---|---|---|---|---|
| | 3V3 | 1 | 2 | 5V | | |
| I2C SDA | GPIO2 | 3 | 4 | 5V | | |
| I2C SCL | GPIO3 | 5 | 6 | GND | | |
| | GPIO4 | 7 | 8 | GPIO14 | UART TX | |
| | GND | 9 | 10 | GPIO15 | UART RX | |
| | GPIO17 | 11 | 12 | GPIO18 | PCM CLK | PWM0 |
| | GPIO27 | 13 | 14 | GND | | |
| | GPIO22 | 15 | 16 | GPIO23 | | |
| | 3V3 | 17 | 18 | GPIO24 | | |
| SPI MOSI | GPIO10 | 19 | 20 | GND | | |
| SPI MISO | GPIO9 | 21 | 22 | GPIO25 | | |
| SPI SCLK | GPIO11 | 23 | 24 | GPIO8 | SPI CE0 | |
| | GND | 25 | 26 | GPIO7 | SPI CE1 | |
| I2C ID EEPROM | GPIO0 | 27 | 28 | GPIO1 | I2C ID EEPROM | |
| | GPIO5 | 29 | 30 | GND | | |
| | GPIO6 | 31 | 32 | GPIO12 | PWM0 | |
| PWM1 | GPIO13 | 33 | 34 | GND | | |
| PWM1 PCM FS | GPIO19 | 35 | 36 | GPIO16 | | |
| | GPIO26 | 37 | 38 | GPIO20 | PCM DIN | |
| | GND | 39 | 40 | GPIO21 | PCM DOUT | |

Source: randomnerdtutorials.com

# Un circuit simple

*Une entrée et une sortie digitales*




Contact par ligne de 5

Attention au sens de la LED!

# Un circuit simple

*Une entrée et une sortie digitales*

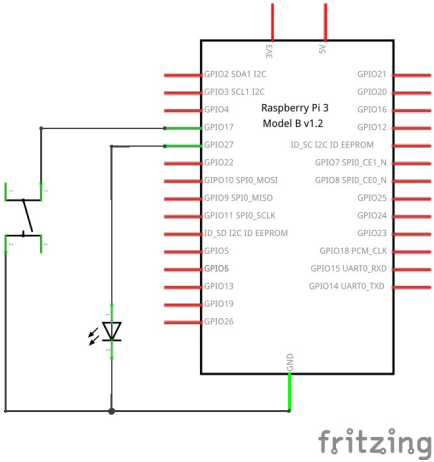



Schéma du circuit précédent

Le bouton (à gauche):

- par défaut: pas de contact
- pressé: contact entre GPIO17 et GND

La LED (à droite):

- allumée si GPIO27 est à 3.3V
- éteinte si GPIO27 est à 0V

# Python

*Un langage de programmation*

Haut niveau

- plus simple à aborder
- prototypage rapide

Interprété

- pas de compilation
- multiplateforme

À usage général

- très flexible
- large écosystème de librairies

# Premier programme Python

*Bases de programmation*

**`nano`** *`hello.py`*: créer un premier code Python

```
print("Hello, world!")
```

**`python`** *`hello.py`*: exécuter le code

Sur Raspberry Pi OS, on peut utiliser *Geany* pour écrire du code

Attention! Les indentations (espaces **OU** tabs) sont cruciales en Python!

# Retenir l’information: les variables

```python
# Ceci est un commentaire, il est ignore par Python
# Les variables permettent de stocker de l’information

# Par exemple, un nombre
nombre = 42.0

# Ou du texte, entre " ou ’
texte = "Voici du texte"

# Ou une valeur logique
vrai = True
faux = False

# On peut effectuer des operations et modifier les variables
nombre = (nombre - 5) * 3 / 2 + 1
```

# Regrouper des variables: les listes et dictionnaires

```
# Une liste peut contenir plusieurs variables differentes
liste = [42, "texte", True]

# On utilise [] pour acceder a un element, en partant de 0
print("liste[1] =", liste[1])

# On peut ajouter des elements a la fin de la liste
liste.append(3.14)
print("liste apres append:", liste)

# Et mesurer sa longueur
longueur = len(liste)
print("longueur de la liste:", longueur)

# Un dictionnaire associe chaque element a une cle
dictionnaire = {"univers": 42, "pi": 3.14}
print("dictionnaire['univers'] =", dictionnaire["univers"])
```

# Faire des choix: les conditions

```python
compteur = 0

# Les conditions permettent de faire des choix dans le programme
if compteur > 5:
    print("Compteur est strictement plus grand que 5")

# elif est une seconde condition, si la premiere est fausse
elif compteur == 5:
    print("Compteur vaut 5")

# on peut combiner les operateurs logiques and, or, not
elif (False or True) and not True:
    print("Ceci ne se produira pas")

#else est la condition par defaut, si le autres sont fausses
else:
    print("Compteur <= 5")
```

# Répéter des opérations: les boucles

```python
compteur = 0

# Les boucles while tournent tant que la condition est vraie
while compteur < 10:
    print("Compteur:", compteur)
    compteur = compteur + 1

# Les boucles for iterent sur une sequence d'elements
for index in range(5):
    print("index:", index)

# On peut "casser" une boucle en avance avec break
while True:
    print("Ceci tourne pour toujours")
    compteur = compteur + 1

    if compteur > 20:
        break
```

# Réutiliser du code: les fonctions et librairies

```python
# Les fonctions peuvent etre appelees avec des arguments
# syntaxe: def nom_fonction(arguments):
def ajouter(a, b):
    print("Addition de", a, "et", b)

    # Et retourner une valeur
    return a + b

resultat = ajouter(3, 4)
print("Resultat:", resultat)

# On peut importer des librairies pour utiliser des fonctions existantes
import math
resultat = math.sqrt(2)
print("Racine carre de 2:", resultat)
```

# `gpiozero`

*Une librairie simple pour contrôler les GPIO*

`gpiozero` permet de contrôler les GPIO en Python facilement:

- Pré-installé sur Raspberry Pi OS
- API simple et intuitive
- Documentation complète et claire (`https://gpiozero.readthedocs.io/`)
- Nombreux périphériques inclus (LED, bouton, moteurs, capteurs, etc.)

Objectifs du jour:

1. Contrôler une LED
2. Utiliser un bouton

# Contrôler une LED

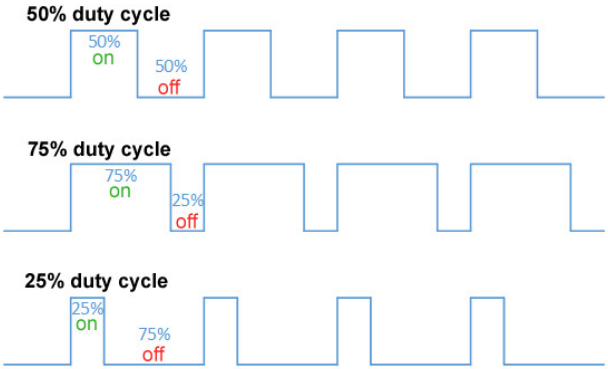
*Allumer et éteindre*

```python
# On n'importe que ce dont on a besoin ici
from gpiozero import LED
from time import sleep

# Creer un objet LED sur le port GPIO27 (pin 13)
led = LED(27)

# Faire clignoter la LED 10 fois
for i in range(10):
    led.on()  # Allumer la LED
    sleep(0.5)  # Attendre 0.5 secondes
    led.off()  # Eteindre la LED
    sleep(0.5)
```

# Comment varier la luminosité d’une LED controlée par une sortie digitale?

# Tricher avec le PWM

*Pulse Width Modulation*




Source: commons.wikimedia.org

Alternance haute vitesse 1/0

Trop rapide pour nos yeux
$\rightarrow$ impression d'analogique

Varier la largeur pour "l'intensité"

# Contrôler une LED

*Varier la luminosité avec le PWM*

```
from gpiozero import PWMLED
from time import sleep

# Creer un objet PWMLED sur le port GPIO27 (pin 13)
led = PWMLED(27)

# Faire clignoter la LED 10 fois
for i in range(10):
    led.value = 0.7 # Allumer la LED a 70% de luminosite
    sleep(0.5)  # Attendre 0.5 secondes
    led.value = 0.2 # Diminuer la luminosite a 20%
    sleep(0.5)

led.value = 0.0 # Eteindre la LED
```

Soyez créatifs, imaginez des motifs de clignotement (rampe, sinus, ...)!

# Utiliser un bouton

*Lire la valeur manuellement*

```python
from gpiozero import Button
from time import sleep

# Creer un objet Button sur le port GPIO17 (pin 11)
button = Button(17)

# Lire la valeur du bouton chaque seconde
for i in range(10):
    if button.is_pressed:
        print("Le bouton est appuye!")
    else:
        print("Le bouton n’est pas appuye.")
    sleep(1)
```

# Utiliser un bouton

*Interrompre le programme quand on appuie*

```python
from gpiozero import Button
from time import sleep

# Creer un objet Button sur le port GPIO17 (pin 11)
button = Button(17)

# Creer une fonction a executer quand on appuie sur le bouton
def on_press():
    print("Bouton appuye!")

# Associer la fonction a l'evenement "when_pressed" du bouton
button.when_pressed = on_press

input("Appuyez sur Enter pour terminer le programme...")
```

# Exercice de synthèse

*Contrôler la LED avec le bouton*

Combiner les éléments `LED/PWMLED` et `Button`

Objectifs:

1. allumer la LED quand le bouton est appuyé
2. éteindre la LED quand le bouton est appuyé à nouveau
3. répéter

Bonus:

utiliser le PWM pour allumer et éteindre la LED progressivement

faire clignoter la LED tant que le bouton est appuyé

# Le capteur DHT22

*Une mini-station météo*

Mesure température (± 0.5°C) et d'humidité (± 2%)

Brancher le DHT22 au GPIO:

- `+` → 5V
- `out` → GPIO 4 (pin 7)
- `-` → GND

Utilise un protocole 1-fil propriétaire
→ nécessite une librairie dédiée

# Installer des librairies Python

## *`pip` et `venv`*

**`pip`** est le gestionnaire de paquets Python

- **`pip`** *`install librairie`*: installer une librairie
- **`pip`** *`list`*: lister les librairies installées
- **`pip`** *`uninstall librairie`*: désinstaller une librairie

`venv` est un module Python pré-installé pour créer des environnements virtuels

- isoler les dépendances d'un projet
- utiliser différentes versions de python ou de librairies

# Mise en place pour le DHT22

*Création d'un environnement virtuel et installation de librairies*

Se déplacer dans un dossier de travail, puis:

1. Créer un environnement virtuel nommé `pienv`
   **`python`** *`-m venv pienv --system-site-packages`*
2. Vérifier que l'environnement virtuel a été crée
   **`ls`** $\rightarrow$ chercher un dossier `pienv`
3. Activer l'environnement virtuel
   **`source`** *`pienv/bin/activate`*
4. Installer la librairie
   **`pip`** *`install adafruit-circuitpython-dht`*

# Lire les mesures du DHT22

```
import adafruit_dht
import board

# Creer un objet DHT22 sur le port GPIO4 (pin 7)
dht = adafruit_dht.DHT22(board.D4)

# Lire la temperature et l'humidite
print("Temperature:", dht.temperature, "C")
print("Humidite:", dht.humidity, "%")
```

Exercices:

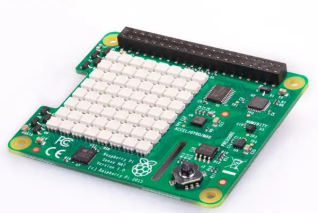
- afficher les mesures toutes les secondes
- afficher les mesures lorsqu'on appuie sur le bouton
- allumer la LED si la température dépasse un seuil

# Les HATs

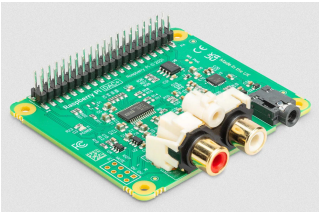
*Bien plus que des chapeaux*

Hardware Attached on Top (HAT): extensions matérielles pour Raspberry Pi

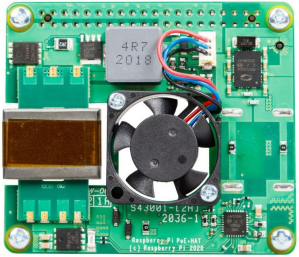
- se connectent sur les broches GPIO
- offrent des fonctionnalités variées



Sense HAT



DAC+ HAT




PoE+ HAT